A Machine-Checked Model of the elizaOS Memory Subsystem, and the Over-Constraint Trichotomy of Agent Retrieval

Formalised in Lean 4 with Mathlib

Abstract

We give a Lean 4 formalisation of the memory subsystem of the open-source agent framework elizaOS/eliza: memory records, the database-adapter operations (`getMemories`, `searchMemories`, `createMemory`, `createMemory` with de-duplication, `removeMemory`, `removeAllMemories`, `countMemories`), and the retrieval constraints these operations impose. The central result is an *over-constraint trichotomy*: eliza's three retrieval constraints — the result *count*, the similarity *threshold*, and the *room* scope — have exactly three different order-theoretic characters (monotone budget, antitone gate, local partition), and each of the three is *independently* a source of incompleteness, witnessed by three machine-checked counterexamples in which a memory satisfying every other constraint is provably dropped. Around this core the development builds a verified consistency gateway, an all-or-nothing transaction layer with an audit-receipt chain, a self-hosting "inside-out" reading in which the agent's own operations are stored as memories and run by an interpreter that has a fixed point, and several structural layers (graded and filtered memories with a Hilbert function and Poincaré polynomial, content-addressed storage, model-interchange round trips, and an arithmetisation pipeline). The whole development consists of 109 Lean modules, roughly 953 theorems and 617 definitions over about 17,600 lines; it compiles against Lean 4.28.0 with Mathlib and contains no `sorry` or `admit`. Every theorem below is stated *with its proof*: the argument in the text is the argument the Lean script performs, the scripts of the central results are reproduced verbatim in Appendix A, and §13 records exactly what the kernel checks and on which axioms each result depends.

Contents

1 Introduction

1.1 The object of study

An LLM agent framework is, operationally, a memory system with a language model attached. In elizaOS/eliza the memory subsystem is the database-adapter interface: a memory record carries an identifier, an entity, an agent, a room, textual content, an embedding vector, a creation timestamp and a uniqueness flag, and the adapter exposes a handful of operations over a table of such records. In TypeScript this is (abridged from `packages/core/src/types.ts`):

```
interface Memory {
  id?: UUID; entityId: UUID; agentId?: UUID; createdAt?: number;
  content: Content; embedding?: number[]; roomId: UUID; unique?: boolean;
}

getMemories({ roomId, count, ... }): Promise<Memory[]>
searchMemories({ embedding, match_threshold, count, roomId, ... }): Promise<Memory[]>
createMemory(memory, tableName, unique?): Promise<UUID>
removeMemory(memoryId, tableName): Promise<void>
removeAllMemories(roomId, tableName): Promise<void>
countMemories(roomId, ...): Promise<number>
```

Everything an eliza agent "knows" at inference time is what these calls return. The returned set is not the set of relevant memories: it is the set of relevant memories that survive three filters, applied in a fixed order — a room scope, a similarity threshold, and a hard cap on the number of results. Those three filters are the "specific topics that are overly constrained" of the title: they are what makes retrieval cheap, and they are exactly what makes it lossy.

1.2 What this paper contributes

The contribution is a faithful, executable, machine-checked model of that subsystem together with a precise account of its loss of information.

1. **A model** (§2). Each record field and each adapter operation is transcribed into Lean 4. Embeddings and similarities are rational (`ℚ`) rather than floating point, so the model is fully computable and concrete instances are decidable. Ordering ("newest first", "most similar first") is modelled with `List.mergeSort`, matching the `ORDER BY \ldots \ DESC` of the SQL adapters.

2. **Soundness, boundedness, ranking, completeness** (§3). Every returned memory is really in the store, in the room, and over the threshold; the result is never longer than the requested count; the result is sorted by decreasing similarity; and, when the cap is at least the number of candidates, *every* relevant memory is returned.

3. **The over-constraint trichotomy** (§4). The dependence of the answer on each of the three constraints is characterised, and each constraint is shown to be an independent cause of information loss.

4. **A verified operational envelope** (§6–§8): a consistency gate that no mutation can violate, an all-or-nothing batch commit, a normal form for transaction histories modulo empty boundaries, and an audit chain of receipts that reconstructs the history.

5. **Unifications** (§9–§11): several places where a construction forced by the eliza data model coincides with a standard mathematical structure — a self-hosting interpreter with a fixed point, a filtration with an associated graded object and Poincaré polynomial, a Galois connection between identity regimes, a Pareto frontier, a content-addressed store, a model-interchange equivalence, and an arithmetisation pipeline.

1.3 Method and standard of evidence

Everything asserted below is a Lean declaration in the accompanying project and is checked by the Lean 4 kernel. Statements about *eliza* (as opposed to statements about the model) are not machine-checked and cannot be: the bridge from TypeScript to Lean is a human transcription, recorded in the module documentation. What the kernel guarantees is that the model has the stated properties; what the reader must judge is that the model is the right one. We have kept the model deliberately small and literal to make that judgement easy.

Counterexample statements are existential and are discharged by evaluation (`native_decide`) on explicit finite data, or by `decide` where the search space allows; they are genuine witnesses, not vacuous truths. No axioms beyond `propext`, `Classical.choice` and `Quot.sound` are involved, together with `Lean.ofReduceBool` and `Lean.trustCompiler` in the counterexamples discharged by `native_decide`.

2 The core model

2.1 Records and stores

```
abbrev UUID := Nat
abbrev Embedding := List ℚ

structure Memory where
  id : UUID
  entityId : UUID
```

```
    agentId : UUID
    roomId : UUID
    content : String
    embedding : Embedding
    createdAt : Nat
    unique : Bool
  deriving DecidableEq

  abbrev Store := List Memory
  abbrev Similarity := Embedding → Embedding → ℚ
```

A `Store` is a list rather than a set: the adapter's table is ordered, insertion is observable, and duplicate rows are possible unless de-duplication is requested. The similarity function is a parameter, not a fixed formula; eliza uses cosine similarity, and the development supplies `dotProduct` as a concrete computable instance with `dotProduct_comm` proved for equal-length vectors. Keeping `sim` abstract means the retrieval theorems hold for *any* scoring function, which is the honest statement: none of them depends on properties of cosine similarity.

2.2 The adapter operations

```
  def getMemories (s : Store) (room : UUID) (count : Nat) : List Memory :=
    ((s.filter (fun m => m.roomId == room)).mergeSort
        (fun a b => decide (b.createdAt ≤ a.createdAt))).take count

  def searchCandidates (s : Store) (sim : Similarity) (q : Embedding)
      (room : UUID) (threshold : ℚ) : List Memory :=
    (s.filter (fun m => m.roomId == room && decide (threshold ≤ sim q m.embedding))).mergeSort
        (fun a b => decide (sim q b.embedding ≤ sim q a.embedding))

  def searchMemories (s : Store) (sim : Similarity) (q : Embedding)
      (room : UUID) (threshold : ℚ) (count : Nat) : List Memory :=
    (searchCandidates s sim q room threshold).take count
```

The split of `searchMemories` into `searchCandidates` followed by `take count` is the pivot of the whole analysis: it separates the two *gates* (room, threshold), which decide relevance, from the *budget* (count), which merely truncates a ranking. Mutations are equally literal:

```
  def createMemory (s : Store) (m : Memory) : Store := m :: s

  def hasNearDuplicate (s : Store) (sim : Similarity) (m : Memory) (threshold : ℚ) : Bool :=
    s.any (fun m' => m'.roomId == m.roomId && decide (threshold ≤ sim m.embedding m'.embedding))

  def createMemoryUnique (s : Store) (sim : Similarity) (m : Memory) (threshold : ℚ) : Store :=
    if hasNearDuplicate s sim m threshold then s else m :: s

  def removeMemory (s : Store) (id : UUID) : Store := s.filter (fun m => m.id != id)
  def removeAllMemories (s : Store) (room : UUID) : Store := s.filter (fun m => m.roomId != room)
  def countMemories (s : Store) (room : UUID) : Nat := (s.filter (fun m => m.roomId == room)).length
```

3 Soundness, boundedness, ranking, completeness

Theorem 3.1 (Retrieval soundness)**.** *For all* `s, sim, q, room, threshold, count` *and every* `m ∈ searchMemories s sim q room threshold count: m ∈ s, m.roomId = room,` *and* `threshold ≤ sim q m.embedding.`

Proof. Unfold: `searchMemories s sim q room threshold count` is `((s.filter P).mergeSort R).take count` with `P m = (m.roomId == room && decide (threshold ≤ sim q m.embedding))` and `R` the comparison by decreasing similarity. Membership in a `take` implies membership in the underlying list (`List.mem_of_mem_take`); `mergeSort` returns a permutation of its input, so `List.mem_mergeSort` transports membership back to `s.filter P`; and `List.mem_filter` splits that into `m ∈ s` together with `P m = true`, whose two Boolean conjuncts are exactly `m.roomId = room` and `threshold ≤ sim q m.embedding` after `decide`-elimination. The `getMemories` statements are the same three steps with `P m = (m.roomId == room)`. Note that no property of `sim` is used anywhere: soundness is structural. □

Lean: `searchMemories_subset`, `searchMemories_mem_room`, `searchMemories_mem_threshold`; and correspondingly `getMemories_mem_room`, `getMemories_subset` for the recency query. Nothing is invented by retrieval, and nothing outside the declared scope leaks in.

Theorem 3.2 (Boundedness and ranking)**.** *`(searchMemories s sim q room threshold count).length ≤ count`, and the result is `List.Pairwise` sorted by decreasing similarity to `q`. Likewise `getMemories` is bounded by its count and pairwise sorted by decreasing `createdAt`.*

Proof. Boundedness is `(l.take n).length = min n l.length ≤ n`. For the ranking, `List.pairwise_mergeSort` gives `List.Pairwise (fun a b => decide (sim q b.embedding ≤ sim q a.embedding) = true)` for the sorted candidate list; its two side conditions — totality and transitivity of the comparison — hold because the comparison is the pullback along `m ↦ sim q m.embedding` of the linear order of ℚ. A `take` is a sublist (`List.take_sublist`) and `List.Pairwise` is inherited by sublists, so the truncated result is still pairwise ordered; `List.Pairwise.imp` then strips the `decide`. For `getMemories` replace the score by `createdAt` and ℚ by ℕ. □

Lean: `searchMemories_length_le`, `searchMemories_sorted`, `getMemories_length_le`, `getMemories_sorted`. The ranking statement is the one that justifies calling the cap a *top-k*: the memories the cap discards are exactly the lowest-scoring candidates, never an arbitrary subset.

Theorem 3.3 (Exact candidates)**.**

```
theorem mem_searchCandidates_iff (s : Store) (sim : Similarity) (q : Embedding)
    (room : UUID) (threshold : ℚ) {m : Memory} :
    m ∈ searchCandidates s sim q room threshold ↔
      m ∈ s ∧ m.roomId = room ∧ threshold ≤ sim q m.embedding
```

Proof. Both directions at once, by unfolding and composing three equivalences: `m ∈ l.mergeSort R ↔ m ∈ l` (`List.mem_mergeSort`, since `mergeSort` permutes), `m ∈ l.filter P ↔ m ∈ l ∧ P m = true` (`List.mem_filter`), and `decide`-elimination of the Boolean conjunction `m.roomId == room && decide (threshold ≤ sim q m.embedding)` into `m.roomId = room ∧ threshold ≤ sim q m.embedding`. □

This biconditional is the precise content of the two gates, and every later result about the gates is a two-line consequence of it. Note what it does *not* mention: the count. Relevance, in eliza, is a property of a memory; being returned is not.

Theorem 3.4 (Completeness below the cap)**.** *If `(searchCandidates s sim q room threshold).length ≤ count`, then every `m ∈ s` with `m.roomId = room` and `threshold ≤ sim q m.embedding` lies in `searchMemories s sim q room threshold count`.*

Proof. `List.take_of_length_le` says `l.length ≤ n → l.take n = l`. Applied to the candidate list this is precisely `searchMemories_eq_searchCandidates_of_le`: under the hypothesis the cap is the identity, so `searchMemories` *is* `searchCandidates`. The conclusion is then the right-to-left direction of Theorem 3.3. For `getMemories` the same argument applies once the hypothesis `countMemories s room ≤ count` is read as a bound on the length of `s.filter (fun m => m.roomId == room)`, which is its definition. □

Lean: `searchMemories_complete`; `getMemories_complete` is the analogue for recency retrieval, and `searchMemories_eq_searchCandidates_of_le` strengthens the statement to an equality of lists — when the budget is generous the cap is not merely harmless, it is invisible.

4 The over-constraint trichotomy

The three constraints are usually discussed as though they were interchangeable knobs. They are not: they differ in their order-theoretic character, and therefore in how a system should reason about them.

Theorem 4.1 (Trichotomy)**.** *1.* The count is a monotone budget. *If `count ≤ count'` then every result at `count` is a result at `count'` (`searchMemories_count_monotone`, `getMemories_count_monotone`).*

2. The threshold is an antitone gate. *If `threshold ≤ threshold'` then every candidate at `threshold'` is a candidate at `threshold` (`searchCandidates_threshold_antitone`); the candidate count is likewise antitone (`searchCandidates_length_threshold_antitone`).*

3. The room is a local partition. *For `room ≠ room'`,*

```
searchMemories (removeAllMemories s room') sim q room threshold count
  = searchMemories s sim q room threshold count
```

(`searchMemories_room_local`): deleting an entire foreign room changes nothing about the query, on the nose, as lists.

Proof. Write C for `searchCandidates s sim q room threshold`.

(i) If `m ∈ C.take count` then `m = C.get n` for some `n < min count C.length`; since `count ≤ count'` the very same index satisfies `n < min count' C.length` and picks out the same element, so `m ∈ C.take count'`. (Concretely: `C.take count` is a prefix of `C.take count'`.) The `getMemories` case is identical.

(ii) By Theorem 3.3, `m ∈ C(threshold')` gives `m ∈ s`, `m.roomId = room` and `threshold' ≤ sim q m.embedding`; transitivity with `threshold ≤ threshold'` yields `threshold ≤ sim q m.embedding`, and the same theorem read in the other direction puts `m ∈ C(threshold)`. For the statement about lengths, `mergeSort` preserves length, so it suffices that the length of `s.filter` is monotone in the predicate — an induction on the store using `List.filter_cons`, since raising the threshold can only turn a kept row into a dropped one.

(iii) `removeAllMemories s room'` is `s.filter (fun m => m.roomId != room')`, so the left-hand candidate list filters `s` by that predicate and then by `fun m => m.roomId == room && decide (threshold ≤ sim q m.embedding)`. `List.filter_filter` merges the two into their conjunction. For `room ≠ room'` the extra conjunct is implied by `m.roomId == room`, so the merged predicate is pointwise equal to the right-hand one; equal predicates give equal filtered lists, hence equal `mergeSort`s and equal `take`s. The equality is on the nose, as lists — not up to permutation or set equality. □

Monotone, antitone, local: raising the budget can only help, raising the gate can only hurt, and the room is invisible in both directions. The three are logically independent, and the development proves this in the strongest available form — by exhibiting, for each constraint separately, a store in which *that constraint alone* destroys a relevant answer.

Theorem 4.2 (Three independent over-constraint witnesses)**.** *1.*

`searchMemories_overconstrained_drops_relevant`: there is a store, query and `count ≥ 1` such that the result has length exactly `count` (the budget is saturated, so this is not the degenerate `count = 0` case) and yet a memory of the queried room that clears the threshold is not returned.

2. `searchMemories_overconstrained_drops_by_threshold`: *there is a store and query with a* generous *count (so retrieval is complete relative to its candidates) in which a memory of the queried room is dropped solely because its similarity falls below the threshold.*

3. `searchMemories_overconstrained_drops_by_room`: *there is a store and query with a generous count in which a memory that clears the threshold is dropped solely because it lives in another room.*

Proof. Three explicit finite stores. Write a for the row `⟨1,0,0,7,"a",[1],0,false⟩` (identifier 1, room 7, `createdAt` 0), b for `⟨2,0,0,7,"b",[1],1,false⟩` (identifier 2, room 7) and a' for `⟨1,0,0,8,"a",[1],0,false⟩` (room 8).

(1) The budget. Take $s = [a, b]$, `sim` the constant 1, `q = [1]`, `room = 7`, `threshold = 0`, `count = 1`. Both rows clear both gates, so the candidate list has length 2; the ranking key is constant, and `mergeSort` is stable, so the candidate list is $[a, b]$ in store order. Hence `searchMemories … 1` is $[a]$, of length 1, which is exactly `count` — the budget is saturated — while $b \in s$ satisfies `b.roomId = 7` and `0 ≤ sim q b.embedding` and yet $b \notin [a]$.

(2) The gate. Take $s = [a]$, `sim` the constant 0, `threshold = 5`, `count = 10`. The threshold filter deletes the only row, so the candidate list is empty; its length 0 is below the cap (so the cap is provably not the culprit), and yet $a \in s$ lies in the queried room and is not returned.

(3) The partition. Take $s = [a']$, query `room = 7`, `sim` the constant 10, `threshold = 5`, `count = 10`. Now the similarity clears the gate and the cap is generous, but the room filter deletes the row.

Each claim is a finite computation over a store of at most two rows, and is discharged by evaluating the definitions of `searchMemories`, `searchCandidates` and `List.mergeSort` inside the kernel. In particular these three witnesses use no compiled evaluation: they depend only on `propext`, `Classical.choice` and `Quot.sound` (see §13). □

Each witness is a two-row (or one-row) store, checked by evaluation. Their point is methodological: eliza's retrieval is not "approximately complete"; it is complete exactly on the region carved out by Theorem 3.3 and the budget, and outside that region the failure is not statistical but structural. Any downstream claim of the form "the agent considered everything it knew about X" is false unless the room, the threshold and the count are all discharged — and the three must be discharged by three different arguments, because they fail in three different ways.

Remark 4.3. The trichotomy also tells one how to *repair* each constraint. A monotone budget can be repaired by paging (raise the cap, keep the earlier results, they persist by monotonicity). An antitone gate can be repaired by lowering it and re-ranking (the old answers survive). A partition cannot be repaired by tuning at all; it needs a different query — which is exactly why the development adds an explicit multi-room retrieval operation (§5) rather than a wider room.

5 The extended retrieval surface

Real adapters scope by *table* as well as room, accept time windows, and offer a multi-room query. `TableScopedRetrieval` models this:

```
def getMemoriesWindow (s : TStore) (table : String) (room : UUID)
    (start finish : Option Nat) (count : Nat) : List Row
def getMemoriesByRoomIds (s : TStore) (table : String) (rooms : List UUID)
    (count : Nat) : List Row
```

with the same discipline of results: soundness in each coordinate (`getMemoriesWindow_mem_table`, `_mem_room`, `_mem_window`, `_subset`), boundedness (`getMemoriesWindow_length_le`),

recency ordering (getMemoriesWindow_sorted), completeness under a generous cap (getMemoriesWindow_complete), and a fresh over-constraint witness for the new axis: getMemoriesWindow_overconstrained_window exhibits a memory dropped solely by the time window. The multi-room query satisfies getMemoriesByRoomIds_mem_rooms — its results lie in the *union* of the requested rooms — which is the intended weakening of the room partition, made explicit rather than smuggled in.

MemoryLaws and DeduplicationLaws record the remaining algebra: idempotence of room deletion (removeAllMemories_idem), commutativity of deletions across rooms (removeAllMemories_comm), invariance of a room's count under deletion of another room (countMemories_removeAllMemories_ne), monotonicity of the candidate set in the store (searchCandidates_mono), and the de-duplication laws: hasNearDuplicate_iff characterises the duplicate test, createMemoryUnique_of_hasNearDuplicate and createMemoryUnique_of_not_hasNearDuplicate pin the two branches, and createMemoryUnique_idem_of_self_similar shows that a unique insert is idempotent as soon as the similarity function is reflexive at the threshold — the property one wants for a de-duplicator and which is easy to lose by using an unnormalised score.

Two further views of the same store are developed. MemoryGraph builds a SimpleGraph on memories generated by "shares a room or mentions the other's entity" (memoryGraph_adj_iff), and proves that the memories of one room form a clique (room_isClique) and that the graph is monotone in the store (memoryGraph_mono) — the graph-theoretic restatement of room locality. MetaMemory is a catalogue of the memory operations themselves, classified by access kind, with findUsages_sound and findUsages_complete tying the catalogue to the model.

6 Consistency: a gate that no mutation can pass

The model so far is the adapter as written. StateConsistency adds the discipline the adapter lacks. Fix a validity predicate `valid : Memory → Bool` and call a store Consistent when every row satisfies it. Raw insertion does not preserve consistency — raw_create_can_break_consistency exhibits a violation — so the module defines *gated* operations gateCreate, gateUpsert, gateUpdate and an interpreter applyMutations for mutation programs.

Theorem 6.1 (Consistency is unconditional)**.** *applyMutations_consistent: for every mutation program* `mus` *and every store* `s`*, if* `Consistent valid s` *then* `Consistent valid (applyMutations valid s mus)`*; and* `applyMutations_nil_consistent`*: starting from the empty store,* every *program yields a consistent store.*

Proof. `Consistent valid s` is the conjunction of `∀ m ∈ s, valid m = true` and `(s.map (·.id)).Nodup`. Each of the five gateway primitives preserves both conjuncts.

Deletions. removeMemory and removeAllMemories are filters. A filtered list is a sublist, so validity is inherited pointwise; and `(l.filter P).map (·.id)` is a sublist of `l.map (·.id)`, so Nodup is inherited too.

gateCreate. The guard is `valid m && !(s.any (fun m' => m'.id == m.id))`. In the guarded branch the store is `m :: s`: the head is valid by the first conjunct of the guard, and its identifier is absent from `s` by the second, so consing preserves Nodup. In the other branch the store is unchanged.

gateUpsert. In the guarded branch the store is `m :: s.filter (fun m' => m'.id != m.id)`: the filter removes *every* row carrying `m.id`, so again the head's identifier is fresh for the tail, and the tail inherits both conjuncts as a filter of `s`.

gateUpdate. In the guarded branch every row with identifier `m.id` is replaced by `m`. Since the replacement carries that same identifier, the list `map (·.id)` is unchanged pointwise, so Nodup transfers verbatim; validity holds for the replaced rows by the guard and for the untouched rows by hypothesis.

applyMutation_consistent is the five-way case split on the mutation, and applyMutations is a `List.foldl` of applyMutation, so applyMutations_consistent follows by induction on the program,

applying the one-step theorem at each stage. The empty store is consistent (both conjuncts hold vacuously), which gives `applyMutations_nil_consistent` as the instance at `s = []`. □

Remark 6.2. The gate is not vacuous. `raw_create_can_break_consistency` exhibits a consistent one-row store and a second row carrying the same identifier, whose raw insertion violates the primary-key conjunct; `gate_blocks_duplicate` shows the gateway returns the store unchanged on exactly that request.

The second form is the useful one: it is a statement about all possible client behaviour, with no obligation on the client. The gate is also shown to be *transparent* on good input (`gateCreate_eq_of_ok`: a valid row is inserted verbatim) and inert on bad input (`gateCreate_eq_self_of_bad`), so the discipline costs nothing when it is not needed.

7 Transactions, refinement, and normal forms

`TransactionalMemory` lifts the gate from single mutations to batches:

Theorem 7.1 (All or nothing)**.** *`commitBatch_all_or_nothing`: for every `valid`, `s`, `mus`, `commitBatch valid s mus` is* either *the unchecked result of applying the whole batch* or *the original store — there is no third, partially applied outcome. Moreover `commitBatch_consistent` and `commitTransactions_consistent` propagate consistency across whole histories, and `duplicate_batch_rolls_back` exhibits a batch that is rolled back for a genuine reason.*

Proof. By definition `commitBatch valid s mus` is `if Consistent valid (uncheckedBatch s mus) then uncheckedBatch s mus else s`, the decision being taken by `Classical.propDecidable` (hence the definition is `noncomputable`, but total). A case split on that single condition gives the disjunction; there is no third branch, so no partially applied state can ever be exposed. The same case split proves `commitBatch_consistent`: in the *then*-branch the visible store is consistent because that is the branch condition, and in the *else*-branch because it is `s`, consistent by hypothesis. Finally `commitTransactions` is a `List.foldl` of `commitBatch`, so `commitTransactions_consistent` follows by induction over the list of batches. □

`GatewayTransactionRefinement` then relates the two levels. Call a batch `BatchAdmissible` at `s` when every step, applied from the state it actually meets, is individually acceptable — formally, `BatchAdmissible valid s []` is `True` and `BatchAdmissible valid s (mu :: mus)` is `Consistent valid (applyUnchecked s mu) ∧ BatchAdmissible valid (applyUnchecked s mu) mus`.

Theorem 7.2 (Refinement)**.** *`commitBatch_eq_applyMutations_of_admissible`: if `s` is consistent and `mus` is admissible at `s`, then `commitBatch valid s mus = applyMutations valid s mus`.*

Proof. Two lemmas, both by induction on the batch, generalising the store.

(a) Steps agree. If the unchecked result of a single mutation is consistent then the guard performs exactly it (`applyMutation_eq_applyUnchecked_of_consistent`); the proof is a case split on the mutation in which the consistency of the candidate supplies precisely the facts the guard tests — for `create`, that the new row is valid and its identifier does not already occur (else the candidate's identifier list would not be `Nodup`); for `upsert` and `update`, that the new row is valid; for the two deletions, nothing, since they are unguarded. Iterating, `uncheckedBatch s mus = applyMutations valid s mus` for an admissible batch: each step of the fold meets a consistent private state by admissibility, so the induction hypothesis applies at `applyUnchecked s mu`.

(b) The commit fires. An admissible batch has a consistent final candidate (`uncheckedBatch_consistent_of_admissible`: the last state of the fold is consistent by the last conjunct of admissibility, again by induction), so `commitBatch` takes its *then*-branch and returns `uncheckedBatch s mus`, which by (a) is `applyMutations valid s mus`.

The degenerate case is `commitBatch_singleton_eq_applyMutation`, which needs no admissibility: for a one-mutation batch, either the candidate is consistent and (a) applies, or it is

not, in which case `commitBatch` rolls back to `s` and the guard also leaves `s` untouched (`applyMutation_eq_self_of_inconsistent` — note the two deletions can never make a consistent store inconsistent, so an inconsistent candidate must have come from a rejected create, upsert or update). □

This is a refinement theorem in the usual sense: the cheap implementation is observationally indistinguishable from the reference semantics precisely where it is allowed to run. The hypothesis is doing real work: outside admissibility the batch evaluator rolls the whole batch back where the step gate would have skipped only the offending step, so the two layers should not be expected to agree there. (That divergence is a remark about the definitions, not a theorem of the development.)

A large sub-development (23 modules) treats transaction *histories* as data. Empty atomic batches are boundaries with no memory effect, so histories are quotiented by insertion and deletion of empty batches; the quotient has canonical representatives (normalisation), a prefix order, ranks, random access to boundaries, and a full slicing API — `take`, `drop`, `interval`, splitting, splicing, replacement, functoriality of intervals under prefix restriction — all proved to be independent of the chosen boundaries. The point is not the API but the invariance: every operation a log viewer would want is available *on the logical trace*, without ever committing to a physical boundary placement.

8 Audit: receipts that reconstruct the history

`TransactionAuditTrail` and its companions make execution self-documenting. Each transaction emits a receipt; a receipt is `ReceiptSound` when it faithfully records the transition it claims.

Theorem 8.1 (Audit chain)**.** *`executeHistory_audit_chain`: for every validity predicate, initial store and history, the receipts emitted by `executeHistory` form a `ReceiptChain` from the initial to the final store: consecutive receipts compose, each is sound, and the chain's endpoints are the actual endpoints of execution. `ReceiptChain.nil_iff` shows an empty chain cannot hide a change, and `executeHistory_audit_final_consistent` shows the audited final state is consistent.*

Proof. Induction on the history, generalising the store. For the empty history `executeHistory valid s [] = (s, [])` and `ReceiptChain.nil` applies. For `mus :: txs`, execution emits `r = executeWithReceipt valid s mus` and continues from `r.after`. Three facts assemble the chain through `ReceiptChain.cons`: `r.before = s`, which holds definitionally (`executeWithReceipt_before`); `ReceiptSound valid r` (`executeWithReceipt_sound`), proved by a case split on the same consistency test that `commitBatch` uses — on acceptance the receipt's decision is `accepted` and its `after` field is the candidate, on rejection the decision is `rejected` and `after` is `before`; and a chain from `r.after` to the final store, which is the induction hypothesis instantiated at `r.after`. The remaining obligation `r.after = r.after` is `rfl`. `ReceiptChain.nil_iff` is the inversion of the `nil` constructor, and the final-state consistency claim is `executeHistory_final_consistent`, itself the audit-free transaction theorem above. □

Companion modules add a Boolean checker (`TransactionAuditChecker`) that is proved equivalent to the propositional specification, compositionality of verification across concatenated histories, interval recovery, replay, segmentation and windowing. Together these say: the log is not a side effect of execution, it is a proof object for it.

9 Inside out: the agent as memories plus an interpreter

The modules above treat the store as data and the operations as functions. The next step inverts that. `InsideOut` encodes each adapter operation as a *memory row* (`Op.toMemory`), so that a program is a store and the agent is one interpreter.

Theorem 9.1 (Eliza inside out). *theorem eliza_inside_out (sim : Similarity) (s : Store) (prog : List Op) :*

```
Function.Injective Op.toMemory ∧
(∀ o : Op, (Op.toMemory o).toOp? = some o) ∧
decompile (compile prog) = prog ∧
runProgram sim s (decompile (compile prog)) = runProgram sim s prog
```

Proof. `Op` is an inductive type with an `Encodable` instance; `Op.toMemory o` is a fixed default row with `id := Encodable.encode o` and content tag `"@op"`, and `Memory.toOp? m` is `Encodable.decode m.id`. The second conjunct is then `Encodable.encodek`: `decode (encode o) = some o`. Injectivity follows: if `toMemory a = toMemory b`, apply `toOp?` to both sides to get `some a = some b`. For the third, `decompile (compile prog)` is `(prog.map Op.toMemory).filterMap Memory.toOp?`, which `List.filterMap_map` turns into `prog.filterMap (Memory.toOp? ∘ Op.toMemory)`; the composite is `fun o => some o` by the second conjunct, so `List.filterMap_some` returns `prog`. The fourth conjunct is then a rewrite by the third. Observe that the interpreter never leaves the medium: `exec_rows_subset` shows every row an instruction returns was already in the store. □

The encoding is injective, decoding inverts it, and — the operative clause — compiling a program into memories and running the decompiled result computes exactly what running the original program computes. Code and data are the same table.

`ReflectiveKernel` pushes this to a fixed point. The map `reflect`, which replaces a store by the code of the operations that would rebuild it, is idempotent (`reflect_idempotent`), lands in code stores (`isCodeStore_reflect`), and forms a Galois connection with decompilation (`reflect_galoisConnection`).

Theorem 9.2 (A self-hosting kernel exists). *theorem exists_selfHosting_kernel (sim : Similarity) :*

```
∃ S₀ : Store, selfRun sim S₀ = S₀ ∧ IsCodeStore S₀ ∧ decompile S₀ ≠ []
```

Proof. Take `kernelProgram = [Op.get 0 0, Op.count 0]` and `S₀ = compile kernelProgram`. By the round trip of the previous theorem, `decompile S₀ = kernelProgram`; hence `reflect S₀ = compile (decompile S₀) = S₀`, so `S₀` is a code-store, and `decompile S₀ = kernelProgram ≠ []`, so it is not the trivial empty fixed point. For the fixed-point clause, both stored instructions are queries (`Op.isQuery`), and a query has identity store effect (`effect_query`); `runProgram` folds the effects, so by induction on the program (`runProgram_query`) running a read-only program leaves the store untouched. Applying this to the program stored in `S₀` gives `selfRun sim S₀ = S₀`, for every similarity `sim`. □

Remark 9.3. The fixed point is exhibited, not merely asserted to exist, and the argument shows exactly what makes it work: read-only code. A kernel containing a mutation would move the store, and the corresponding fixed-point question is then a genuine recursion-theoretic one rather than a triviality — a natural direction the present development does not settle.

That is: a non-empty store of code which, when run on itself, reproduces itself exactly. The bundled statement `eliza_reflective_kernel` collects idempotence, the round trip, code-ness and the fixed point. The categorical bookkeeping is finished by `card_OpTag = 7`: the operation algebra has exactly seven generators, one per adapter call, and `Op.tag_surjective` shows none is redundant.

10 Structure forced by the data model

10.1 Grading and filtration

If a corpus is loaded into memory with each article's room set to its prerequisite depth, the store is not a bag: it is graded. `GradedMemory` defines the dependency relation, proves that prerequisites strictly lower the grade (`depRel_lowers`, `prereq_memory_lower`), that the graded components are

homogeneous and partition the corpus (`gradeComponent_homogeneous`, `mem_gradeComponent_iff`), and that the partial loads `loadUpTo` are monotone (`loadUpTo_mono`), prerequisite-closed (`loadUpTo_prereq_closed`) and eventually exhaust the corpus (`loadUpTo_eq_loadCorpus`).

Monotone and exhaustive is precisely the data of a filtration, and `FilteredMemory` packages it as one: `corpusFiltration_layer_cover` says a memory is in the corpus iff it is in some layer, and `corpusFiltration_stabilizes` says the filtration is finite, stabilising at the top grade.

10.2 Hilbert function and Poincaré polynomial

Once there is a filtration with a known associated graded object, the numerical invariant follows. `PoincareSeries` defines the Hilbert function `hilbert` (the number of memories in each grade) and the Poincaré polynomial `poincare`, and proves the identity that makes them invariants of the store rather than of the presentation:

Theorem 10.1 (Total dimension)**.** *`poincare_eval_one`: if every article has grade at most `B`, then `(poincare c B).eval 1 = (loadCorpus c).length`. Evaluating the Poincaré polynomial at* 1 *recovers the size of the loaded store.*

Proof. Two steps and an evaluation. First, `loadUpTo c N` is `loadCorpus c` filtered by `m.roomId ≤ N`, and grading the corpus by room partitions it, so counting the filtered list grade by grade gives `(loadUpTo c N).length = ∑ n ∈ Finset.range (N+1), hilbert c n` (`loadUpTo_length`, where `hilbert c n = (gradeComponent c n).length`). Second, if every article has grade at most `B`, the filter at level `B` removes nothing, so `List.filter_eq_self` gives `loadUpTo c B = loadCorpus c` and therefore `(loadCorpus c).length = ∑ n ∈ Finset.range (B+1), hilbert c n` (`loadCorpus_length_eq_sum`). Finally `poincare c B = ∑ n ∈ Finset.range (B+1), C (hilbert c n) * X ^ n`, and evaluation is additive with `X ^ n ↦ 1` at `t = 1` (`Polynomial.eval_finset_sum`), so `(poincare c B).eval 1` is that same sum. □

For the concrete sample corpus the Hilbert vector is computed and proved: `hilbert_mathWiki` gives $[1, 1, 1, 2, 1, 2, 1, 1]$ over grades 0 to 7, with `mathWiki_grade_le_seven` bounding the top grade and `poincare_mathWiki_eval_one` instantiating the theorem above.

10.3 Optimal and functional memories

`OptimalMemory` asks what the smallest store is that can serve a fixed set of canonical contents. Content addressing answers it:

Theorem 10.2 (Canonical stores are optimal)**.** *`canonicalStore_optimal`: for a duplicate-free list `cs` of contents, the content-addressed store `cs.map mkMem` is canonical, serves every content in `cs`, and is no longer than* any *store that serves them all.*

Proof. Recall `contentKey m = m.id`, `CanonicalStore s` is `(s.map contentKey).Nodup`, and `Serves s c` is `∃ m ∈ s, contentKey m = c`. Also `contentKey (mkMem c) = c` by definition.

Canonicity. `(cs.map mkMem).map contentKey = cs`, which is `Nodup` by hypothesis.

Service. For `c ∈ cs.toFinset` the row `mkMem c` lies in `cs.map mkMem` and has address `c`.

Optimality. If `s` serves every content of a finite set `C` then `C ⊆ (s.map contentKey).toFinset`, whence `C.card ≤ (s.map contentKey).toFinset.card ≤ (s.map contentKey).length = s.length` (`serves_card_le`) — one record must be spent per distinct content. Taking `C = cs.toFinset` and using `cs.Nodup` to get `cs.toFinset.card = cs.length = (cs.map mkMem).length`, the content-addressed store meets the lower bound, so no store serving the same contents is shorter. □

`FunctionalMemory` takes the dual view: instead of storing a value, store a generator that recomputes it (`exists_generator_reconstructs`, `getMemories_reconstruct_forced`) — memory as a thunk, with retrieval forcing it.

10.4 Identity regimes, lattices and the Pareto frontier

The content-identity layer (`MultihashCID`, `IdentityGalois`, `RegimeLattice`, `PrimeLattice`, `CRTIdentity`) studies the different notions of "same memory" that a system can adopt — cryptographic digest, basin/bucket, semantic equivalence. Each regime induces a closure operator; `cid_galoisConnection` exhibits the adjunction between direct image and preimage, `cidSaturate` is proved extensive, monotone and idempotent, and `sameCid_equivalence` confirms each regime is an equivalence. The regimes are then compared: `semantic_refines` and the negative results `crypto_not_refines_basin`, `basin_not_refines_crypto`, `crypto_not_refines_semantic`, `basin_not_refines_semantic` show the refinement order is genuinely partial, and `crypto_sup_basin_not_regime` shows the regimes are *not* closed under join — there is no "best of both" identity notion inside the family. `cidClosure_injective` and `regimeClosureEmbedding` embed the regimes order-faithfully into the complete lattice of closure operators.

`ParetoFrontier` formalises the resulting two-objective trade-off (how many classes are distinguished versus how much is compressed): `orbitClassCount_add_compression` is the conservation identity, `orbitClassCount_mono_of_refines` and `compression_anti_of_refines` give the two monotonicities, and `paretoRegimes_eq_universe` concludes that *every* regime is Pareto optimal — the choice of identity notion is a preference, never a mistake. `DiagnosticFunctor` lifts the frontier functorially.

11 Storage, interchange and arithmetisation

Three further stacks complete the development.

DASL: a verified content-addressed store. `DASL` defines well-addressedness (every entry stored under the address of its content) and proves `address_injective`, preservation under insertion (`insert_wellAddressed`) and under transfer between stores (`transfer_wellAddressed`). Successive rungs add garbage collection over the store's reachability graph (`DASLGC`), verified diff/patch change control (`DASLPatch`), and a reflective tower: values, schemas, proofs and meta-proofs are internalised as data with checkers proved sound and complete against their propositional specifications (`DASLReflect`, `DASLSchema`, `DASLProof`, `DASLMetaProof`).

Model interchange. `MOFXmi` constructs an equivalence `mofXmiEquiv` between an MOF model tree and its XMI serialisation, with both round trips proved (`fromXmi_toXmi`, `toXmi_fromXmi`) and predicate transport across it (`transport`, `satisfies_toXmi`) — an identification strong enough to move properties, stated honestly as an equivalence rather than as univalence, which Lean's type theory does not provide. The emoji-UML stack (14 modules) then supplies a concrete surface language: a lexer, a deterministic finite automaton for validity, an MOF semantics, forward and reverse pipelines with checked source/XMI round trips, and an algebra of diagram composition that is associative, cancellative, positively graded and non-periodic under iteration.

zkPoP: arithmetisation end to end. The `ZkPoP` namespace (9 modules, 80 theorems) formalises a pipeline from a small typed object language to field constraints and back: `Core` gives types, terms, type inference and evaluation with `Tm.eval_of_hasTy` (type soundness) and `Ty.flat_eq_iff_norm_eq` (arithmetisation is faithful exactly modulo index normalisation); `Poly` gives polynomial systems, Tseitin flattening with `flatten_sound/flatten_complete`, and bit-decomposition range checks (`isNatF_of_bits`); `Universal` encodes circuits as code words and builds a universal checker circuit with `univCircuit_sat_iff` and `univCircuit_self_host`; `Crypto` proves Schnorr completeness and special soundness over Pedersen commitments (`schnorr_complete`, `schnorr_special_soundness`, `artifact_hides_witness`); `Shard` proves Reed–Solomon integrity and recovery and separates private from non-private distribution schemes (`erasure_not_private`, `shamir_private`); `Skill` and `Catalog` assemble the end-to-end artifact with

statement binding (`statement_binding`, `skill_end_to_end`, `skill_compose`); and `Decoder` gives a zero-dependency verifier with `qrVerify_eq_true_iff` and `qrVerify_complete`.

12 Size, coverage and how the numbers are checked

Cluster	Modules	Theorems	Definitions
Eliza memory core	16	176	126
Gateway trace algebra	23	141	32
Audit and receipts	19	128	45
Emoji-UML interchange	14	106	48
DASL schema and proof	7	75	84
Structural mathematics	20	236	188
zkPoP pipeline	9	80	87
Total (whole project)	109	953	617

Table 1: Module counts. The first six rows are the clusters certified by `StatusReport`; the totals count every Lean file in the project, including the `ZkPoP` namespace and the status module itself, using the same counting convention (top-level `theorem`/`lemma` and `def`/`abbrev`/`structure`/`inductive` declarations).

The cluster figures are extracted from the sources by a script and are not merely reported: `StatusReport` re-proves their arithmetic in Lean — the cluster totals sum to the reported totals (`modules_sum`, `theorems_sum`, `defs_sum`, `lines_sum`), every cluster is non-empty (`every_cluster_nonempty`), the displayed percentage shares are the correct roundings (`shares_as_displayed`) and are within rounding of 100% (`shares_nearly_total`), and `coverage_complete` certifies that no `sorry`/`admit` placeholder remains. The project compiles under Lean 4.28.0 with Mathlib.

13 Proofs, and what the machine checks

Every theorem above is stated with a proof, and every proof has a machine-checked counterpart: the argument in the text is the argument the Lean script performs, and the script is what the kernel accepts. It is worth being precise about what that guarantees and what it does not.

The trusted base. A completed Lean proof is a term whose type is the theorem; the kernel re-checks that term from the axioms. Printing the axioms of the results above gives, in each case, a subset of `propext`, `Classical.choice` and `Quot.sound` — the three standard axioms of Mathlib's classical foundation. Nothing in the development introduces an axiom of its own, and no proof is left as `sorry`: the project builds under Lean 4.28.0 with Mathlib with neither placeholder nor local axiom.

Concrete computations. Some statements are finite computations rather than arguments — the three over-constraint witnesses of Theorem 4.2, the Hilbert vector `hilbert_mathWiki`, the arithmetic of `StatusReport`. Two ways of discharging such a goal exist in Lean, and they differ in what they trust: `decide` and `simp`/`norm_num` evaluation run inside the kernel, whereas `native_decide` runs compiled code and adds `Lean.ofReduceBool` and `Lean.trustCompiler` to the trusted base. Because the three witnesses carry the paper's negative results, they are proved by kernel evaluation: unfolding `searchMemories`, `searchCandidates` and `List.mergeSort` on stores of at most two rows is small enough for the kernel, so those theorems depend on the three standard axioms only. The

remaining concrete facts — larger evaluations over the sample corpus and the status figures — do use compiled evaluation, and are flagged as such here rather than in a footnote: they are the only statements in the development whose checking trusts the compiler.

What a proof does not say. A machine-checked proof certifies the Lean statement, not the informal reading of it. The gap that remains is the transcription of eliza's TypeScript into the Lean model (§2), which is documented operation by operation but is not itself verified, and cannot be: it is a modelling judgement. Readers who want to audit that judgement should read the definitions, not the proofs — the definitions are where all the modelling risk lives, and they were kept deliberately short and literal for that reason.

14 Discussion

14.1 What "unification" means here

The request behind this development was to unify Lean 4 and eliza "on specific topics that are overly constrained". The unification is not a claim that eliza *is* mathematics; it is the observation that once eliza's constraints are stated precisely, they land on structures that already have names, and the names come with theorems:

eliza construct	mathematical structure
result count cap	monotone budget on a ranked list (top-k truncation)
similarity threshold	antitone gate; sublevel filtration of the score
room scope	partition; locality of the query functor
`unique` insertion	idempotent closure under a reflexive similarity
transaction batch	all-or-nothing refinement of the step semantics
history with empty batches	quotient with canonical normal forms
audit receipts	chain (composable proof object) of a transition
prerequisite-ordered corpus	graded object, filtration, Poincaré polynomial
content addressing	minimal (optimal) canonical store
identity regimes	closure operators, Galois connection, Pareto frontier
operations stored as memories	self-hosting interpreter with a fixed point

Each row is a theorem, not an analogy: the right-hand column is what is proved in Lean about the left-hand column.

14.2 Limitations

The model is a model. Concurrency is absent: the store is a list, mutated sequentially. Embeddings are rational and similarity is an arbitrary function, so nothing here says anything about the geometry of real embedding spaces or about the quality of cosine similarity as a relevance proxy. The correspondence with eliza's TypeScript source is a transcription, documented in each module but not itself machine-checked, and eliza evolves. The structural layers (§10, §11) are developments *suggested* by the memory model rather than descriptions of shipped eliza behaviour, and the module documentation is explicit about which is which. Finally, the counterexamples of Theorem 4.2 establish that information loss *can* occur; they say nothing about how often it does in deployment.

14.3 What a practitioner should take away

Three things. First, retrieval soundness is free and completeness is not: an agent's answer set is exactly characterised by Theorem 3.3 plus a budget, and the gap is real (Theorem 4.2). Second,

the three constraints must be reasoned about differently, because they are monotone, antitone and local respectively (Theorem 4.1); a tuning strategy that treats them uniformly is mis-specified. Third, the properties one actually wants from the write path — state consistency under arbitrary client programs, all-or-nothing batches, an audit log that reconstructs history — are all provable of a small gateway sitting in front of the raw adapter, and cost nothing on well-behaved input.

A The Lean proof scripts

For the reader who wants the proofs as the machine has them, the scripts of the central results are reproduced verbatim. They are complete: each is the entire proof of the named declaration in the project, with no elided steps.

Exact membership (Theorem 3.3).

```
theorem mem_searchCandidates_iff (s : Store) (sim : Similarity) (q : Embedding)
    (room : UUID) (threshold : ℚ) {m : Memory} :
    m ∈ searchCandidates s sim q room threshold ↔
      m ∈ s ∧ m.roomId = room ∧ threshold ≤ sim q m.embedding := by
  unfold searchCandidates
  simp +decide [List.mem_mergeSort]
```

Soundness of the room gate.

```
theorem searchMemories_mem_room (s : Store) (sim : Similarity) (q : Embedding)
    (room : UUID) (threshold : ℚ) (count : Nat) {m : Memory}
    (hm : m ∈ searchMemories s sim q room threshold count) :
    m.roomId = room := by
  contrapose! hm
  intro h
  have := List.mem_mergeSort.mp (List.mem_of_mem_take h)
  aesop
```

The three axes of Theorem 4.1.

```
theorem searchMemories_count_monotone (s : Store) (sim : Similarity) (q : Embedding)
    (room : UUID) (threshold : ℚ) {count count' : Nat} (hle : count ≤ count')
    {m : Memory} (hm : m ∈ searchMemories s sim q room threshold count) :
    m ∈ searchMemories s sim q room threshold count' := by
  unfold searchMemories at *
  rw [List.mem_iff_get] at *
  obtain ⟨n, hn⟩ := hm
  exact ⟨⟨n, by grind⟩, by simp_all +decide⟩

theorem searchCandidates_threshold_antitone (s : Store) (sim : Similarity)
    (q : Embedding) (room : UUID) {threshold threshold' : ℚ}
    (hle : threshold ≤ threshold') {m : Memory}
    (hm : m ∈ searchCandidates s sim q room threshold') :
    m ∈ searchCandidates s sim q room threshold := by
  rw [mem_searchCandidates_iff] at hm ⊢
  exact ⟨hm.1, hm.2.1, le_trans hle hm.2.2⟩

theorem searchMemories_room_local (s : Store) (sim : Similarity) (q : Embedding)
```

```
    (room room' : UUID) (threshold : ℚ) (count : Nat) (hne : room ≠ room') :
    searchMemories (removeAllMemories s room') sim q room threshold count =
      searchMemories s sim q room threshold count := by
  unfold searchMemories searchCandidates removeAllMemories
  rw [List.filter_filter]
  grind
```

The three over-constraint witnesses (Theorem 4.2).

```
theorem searchMemories_overconstrained_drops_relevant :
    ∃ (s : Store) (sim : Similarity) (q : Embedding) (room : UUID) (threshold : ℚ)
      (count : Nat) (m : Memory),
      1 ≤ count ∧ (searchMemories s sim q room threshold count).length = count ∧
      m ∈ s ∧ m.roomId = room ∧ threshold ≤ sim q m.embedding ∧
      m ∉ searchMemories s sim q room threshold count := by
  refine ⟨[(1, 0, 0, 7, "a", [1], 0, false), ⟨2, 0, 0, 7, "b", [1], 1, false)],
    fun _ _ => 1, [1], 7, 0, 1, ⟨2, 0, 0, 7, "b", [1], 1, false), ?_⟩
  simp [searchMemories, searchCandidates, List.mergeSort]

theorem searchMemories_overconstrained_drops_by_threshold :
    ∃ (s : Store) (sim : Similarity) (q : Embedding) (room : UUID) (threshold : ℚ)
      (count : Nat) (m : Memory),
      m ∈ s ∧ m.roomId = room ∧
      (searchCandidates s sim q room threshold).length ≤ count ∧
      sim q m.embedding < threshold ∧
      m ∉ searchMemories s sim q room threshold count := by
  refine ⟨[(1, 0, 0, 7, "a", [1], 0, false)], fun _ _ => 0, [1], 7, 5, 10,
    ⟨1, 0, 0, 7, "a", [1], 0, false), ?_⟩
  norm_num [searchMemories, searchCandidates, List.mergeSort]

theorem searchMemories_overconstrained_drops_by_room :
    ∃ (s : Store) (sim : Similarity) (q : Embedding) (room : UUID) (threshold : ℚ)
      (count : Nat) (m : Memory),
      m ∈ s ∧ m.roomId ≠ room ∧ threshold ≤ sim q m.embedding ∧
      (searchCandidates s sim q room threshold).length ≤ count ∧
      m ∉ searchMemories s sim q room threshold count := by
  refine ⟨[(1, 0, 0, 8, "a", [1], 0, false)], fun _ _ => 10, [1], 7, 5, 10,
    ⟨1, 0, 0, 8, "a", [1], 0, false), ?_⟩
  norm_num [searchMemories, searchCandidates, List.mergeSort]
```

Consistency under arbitrary client programs.

```
theorem applyMutation_consistent (valid : Memory → Bool) (s : Store) (mu : Mutation)
    (h : Consistent valid s) : Consistent valid (applyMutation valid s mu) := by
  cases mu <;>
    [ exact gateCreate_consistent _ _ _ h;
      exact gateUpsert_consistent _ _ _ h;
      exact gateUpdate_consistent _ _ _ h;
      exact removeMemory_consistent _ _ _ h;
      exact removeAllMemories_consistent _ _ _ h ]

theorem applyMutations_consistent (valid : Memory → Bool) (s : Store)
    (mus : List Mutation) (h : Consistent valid s) :
```

```
    Consistent valid (applyMutations valid s mus) := by
  induction' mus using List.reverseRecOn with mus mu ih <;>
    simp +decide [*, applyMutations]
  exact applyMutation_consistent valid _ _ ih
```

The audit chain.

```
theorem executeHistory_audit_chain (valid : Memory → Bool) (s : Store)
    (txs : List (List Mutation)) :
    ReceiptChain valid s (executeHistory valid s txs).1
      (executeHistory valid s txs).2 := by
  induction' txs with mus txs ih generalizing s
  · constructor
  · exact ReceiptChain.cons (executeWithReceipt_before valid s mus)
      (executeWithReceipt_sound valid s mus) (ih _) rfl
```

Code as data, and the self-hosting fixed point.

```
theorem decompile_compile (prog : List Op) : decompile (compile prog) = prog := by
  unfold decompile compile
  rw [List.filterMap_map]
  have h : (Memory.toOp? ∘ Op.toMemory) = fun o => some o := by
    funext o; simp [Function.comp]
  rw [h, List.filterMap_some]

theorem eliza_inside_out (sim : Similarity) (s : Store) (prog : List Op) :
    Function.Injective Op.toMemory ∧
    (∀ o : Op, (Op.toMemory o).toOp? = some o) ∧
    decompile (compile prog) = prog ∧
    runProgram sim s (decompile (compile prog)) = runProgram sim s prog :=
  ⟨toMemory_injective, toOp?_toMemory, decompile_compile prog,
    runProgram_compile sim s prog⟩

theorem kernel_selfRun_fixed (sim : Similarity) : selfRun sim kernelStore = kernelStore := by
  apply selfRun_query_fixed
  rw [kernel_decompile]
  decide

theorem exists_selfHosting_kernel (sim : Similarity) :
    ∃ S₀ : Store, selfRun sim S₀ = S₀ ∧ IsCodeStore S₀ ∧ decompile S₀ ≠ [] :=
  ⟨kernelStore, kernel_selfRun_fixed sim, kernel_isCodeStore, kernel_has_code⟩
```

Total dimension, and optimality of canonical stores.

```
theorem poincare_eval_one (c : WikiCorpus) (B : Nat)
    (hB : ∀ a ∈ c.articles, c.grade a.id ≤ B) :
    (poincare c B).eval 1 = (loadCorpus c).length := by
  unfold poincare
  simp +decide [Polynomial.eval_finset_sum, loadCorpus_length_eq_sum c B hB]

theorem canonicalStore_optimal (cs : List Nat) (hcs : cs.Nodup) :
    CanonicalStore (cs.map mkMem) ∧
    ServesAll (cs.map mkMem) cs.toFinset ∧
```

```
  (∀ s : Store, ServesAll s cs.toFinset → (cs.map mkMem).length ≤ s.length) := by
refine' ⟨_, _, _⟩
· unfold CanonicalStore; simp +decide
  convert hcs using 1
  exact List.ext_get (by aesop) (by aesop)
· intro c hc; simp_all +decide [Serves]
  exact ⟨c, hc, rfl⟩
· intro s hs; have := serves_card_le hs; simp_all +decide
  rwa [List.toFinset_card_of_nodup hcs] at this
```

Availability

All statements cited above are Lean 4 declarations in the accompanying project, under `Request-Project/`. The names given in the text are the actual declaration names; the core of the paper is `RequestProject/ElizaMemory.lean` and `RequestProject/OverConstraint.lean`.